

Mini Project III

Section 1

1. a)

To begin with logistic regression, I took the *Diabetes* dataset and split it into two parts. A random sample of **70%** of the total 2000 rows went into a **training set**, while the other **30%** of rows went into a **test set**.

The logistic regression model featured **Outcome** as the response(binary), with predictors: **Pregnancies, Glucose Levels, Blood Pressure, Skin Thickness, Insulin Levels, BMI, the Diabetes Pedigree Function, and Age**.

The summary for the model can be found in *Figure 1* below.

```
Call:
glm(formula = Outcome ~ Pregnancies.. + Glucose.. + BloodPressure.. +
    SkinThickness.. + Insulin.. + BMI.. + DiabetesPedigreeFunction.. +
    Age.., family = binomial, data = trainData)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-3.1090	-0.7247	-0.4503	0.7872	2.8918

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-7.8831701	0.5089973	-15.488	< 2e-16	***
Pregnancies..	0.1024971	0.0239076	4.287	1.81e-05	***
Glucose..	0.0323992	0.0025872	12.523	< 2e-16	***
BloodPressure..	-0.0086861	0.0037535	-2.314	0.02066	*
SkinThickness..	-0.0017409	0.0049276	-0.353	0.72387	
Insulin..	-0.0007822	0.0006741	-1.160	0.24591	
BMI..	0.0705581	0.0103352	6.827	8.67e-12	***
DiabetesPedigreeFunction..	0.9808873	0.2211039	4.436	9.15e-06	***
Age..	0.0195486	0.0067091	2.914	0.00357	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1792.2 on 1399 degrees of freedom
Residual deviance: 1358.3 on 1391 degrees of freedom
AIC: 1376.3

Number of Fisher Scoring iterations: 5

[1] 0.215

logReg.pred	0	1
0	353	92
1	37	118

Error rate: $(FP + FN) / (TP + TN + FP + FN) = 0.215$
Sensitivity: $TP / (TP + FN) = 0.5619048$
Specificity: $TN / (TN + FP) = 0.9051282$

Figure 1

As seen in Figure 1, notice from the confusion matrix in blue swirls we can conclude what appears in the blue gradient box.

The misclassification rate was 21.5% for the logistic model using all predictors in the dataset.

The Specificity of 90% suggests that this model *correctly identifies the **absence*** of Diabetes in patients *90% of the time*, while the **Sensitivity of 56%** demonstrates that it ***only correctly identifies Diabetes in patients 56% of the time.***

b)

```
#observations
n <- nrow(diabetes)

squaredErrorsLoocv <- numeric(n)

#LOOCV
for (i in 1:n)
{
  #create training/test sets
  trainData <- diabetes[-i, ]
  testData <- diabetes[i, , drop = FALSE]

  #train using logistic regression
  LOOCV <- glm(Outcome ~ Pregnancies.. + Glucose.. + BloodPressure..
               + SkinThickness.. + Insulin.. + BMI..
               + DiabetesPedigreeFunction.. + Age..,
               data = train_data, family = binomial)

  #make predictions for the i-th test case
  prediction <- predict(LOOCV, newdata = testData, type = "response")

  #square the difference, store
  actualOutcomes <- testData$Outcome
  squaredErrorsLoocv[i] <- (prediction - actualOutcomes) ^ 2
}

#Mean Squared Error for LOOCV
LOOCV_MSE <- mean(squaredErrorsLoocv)
```

Figure 2

Using a simple algorithm for **Leave-One-Out-Cross-Validation (LOOCV)** from Figure 2, *I was able to derive a **Mean Squared Error of 0.1544498.***

c)

After writing out an LOOCV algorithm, I compared the results to R's boot library `cv.glm()` function which, according to the R documentation, "[The] function calculates the estimated K-fold cross-validation prediction error for generalized linear models."

The results can be seen below in Figure 3.

```

$call
cv.glm(data = diabetes, glmfit = model, k = n)

$k
[1] 100

$delta
[1] 0.1561932 0.1561841

```

Figure 3

```

$call
cv.glm(data = diabetes, glmfit = model, k = 10)

$k
[1] 10

$delta
[1] 0.1560782 0.1559898

```

Figure 4

Figure 3 shows the `cv.glm()` method with $k = n$ folds. Here, it should equal my LOOCV MSE of 0.1544498 as **they're identical when $K = n$ folds**. I suppose the **difference of 0.002..** between my algorithm and `cv.glm()` could be in how the MSE is derived as I calculated the mean after the summation of components, while `cv.glm()` could divide by $k = n$ after each iteration and sum at the end.

Figure 4 shows a similar MSE calculation derived from the `cv.glm()` K-Folds CV **where $k = 10$** , which is **much less computationally exhaustive** than the LOOCV method I wrote.

2. a)

If we're looking for the *agreement between two methods*, then a nice metric would be to use **Total Deviation Index (TDI) where $TDI = | \text{difference between the two methods} |$**

Ideally, we're wanting the difference between the two methods to be close to zero because zero means that there's no difference between the two methods. To allow this to be nonparametric, we're deriving statistics directly from the data instead of inferring/making assumptions of the underlying distributions in the set.

Allowing $TDI = |D| = \theta$, and then allowing a large probability, p , to represent *the p^{th} quantile* of θ , it's clear that a **smaller θ is desirable** for a few reasons:

- It suggests that the two methods are more likely to produce similar results, indicating **better agreement**.
- It implies that the methods are **more consistent and reliable** in their measurements/predictions.
- The 90th percentile means that 90% of the absolute differences between the observations in the two populations **are less than or equal to θ** .

- A smaller θ here indicates that a **larger proportion of the observations are closer together**.

b)

Since θ is a quantile, a natural estimator for it is the corresponding empirical quantile from the sample dataset. This is easily found in R using the `quantile()` method, as seen at the top of *Figure 5*.

```
#point-estimation
thetaHat <- quantile(oxygenSats$D, 0.9)

# Bootstrap NonParametric #
B <- 1000 #number of samples
bootstrapEstimates <- numeric(B)

for (b in 1:B)
{
  bootstrapSample <- sample(oxygenSats$D, replace = TRUE)
  bootstrapEstimates[b] <- quantile(bootstrapSample, 0.9)
}

#bias
bias <- mean(bootstrapEstimates) - thetaHat

#standard error
sdError <- sd(bootstrapEstimates)

# 95% upper confidence bound
upperBound <- quantile(bootstrapEstimates, 0.95)

cat("\nPoint estimate (theta_hat):", thetaHat, "\n")
cat("Bias:", bias, "\n")
cat("Standard Error:", sdError, "\n")
cat("95% Upper Confidence Bound:", upperBound, "\n")
```

Figure 5

Figure 6 shows the output of the above code in *Figure 5*. Clearly, the point estimate of θ^{hat} shows that **90% of the difference between the pulse oximetry screener and the oxygen saturation monitor falls below 2**.

Meanwhile, the **bias is close to zero, which is ideal**.

The **Standard Error of θ^{hat} is not optimized** as it shows deviance in the estimation of **0.124989**, **but it isn't too high**.

Lastly, the **95% Upper Confidence Bound of 2.2** isn't too far from the **point estimate of 2**, which further supports the agreement between POS and OSM.

```
Point estimate (theta_hat): 2
Bias: 0.00011
Standard Error: 0.1249892
95% Upper Confidence Bound: 2.2
```

Figure 6

As a quick check, *Figure 7* below shows the TDI distribution which easily, naively demonstrates the point estimate accuracy and relevancy and clearly shows that **majority of the differences lie below 2.0**.

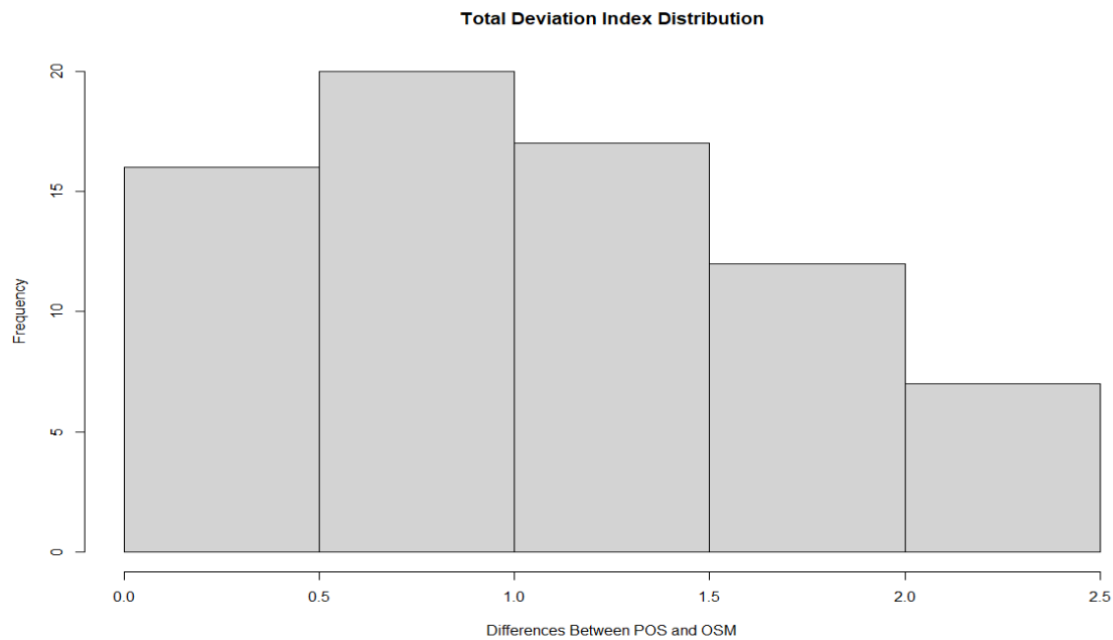


Figure 7

c)

Using the boot method from the boot library, I was able to achieve similar results for the point estimate, bias, and standard error. *Figure 8* shows the code, while *Figure 9* shows the results.

```
#(c)
library(boot)

statistic <- function(data, indices)
{
  quantile(abs(data[indices, 1] - data[indices, 2]), 0.9)
}

bootstrap <- boot(oxygenSats, statistic, R = 1000)

bootstrap
```

Figure 8

ORDINARY NONPARAMETRIC BOOTSTRAP

Call:

```
boot(data = oxygenSats, statistic = statistic, R = 1000)
```

Bootstrap Statistics :

	original	bias	std. error
t1*	2	-0.00267	0.1347239

Figure 9

d) Overall Interpretation:

Based on these results, it appears that POS and OSM methods show a reasonable level of agreement in measuring oxygen saturation. After derivation of the natural point-estimate, with the bias near zero and small standard error, one can conclude that the two methods may be used interchangeably.

3. a) To derive the decision rule for binary classification using LDA:

First verify the equality:

Given:

$$X | (Y = 1) \sim N(\mu_1, \Sigma)$$

$$X | (Y = 2) \sim N(\mu_2, \Sigma)$$

$$P(Y = 1) = \pi_1$$

$$P(Y = 2) = \pi_2$$

Using Baye's Theorem:

$$P(Y = 1 | X) = P(X | Y = 1) * P(Y = 1) / P(X)$$

$$P(Y = 2 | X) = P(X | Y = 2) * P(Y = 2) / P(X)$$

Divide the two equations:

$$P(Y = 1 | X) / P(Y = 2 | X) = [P(X | Y = 1) * P(Y = 1)] / [P(X | Y = 2) * P(Y = 2)]$$

Simplify:

$$P(Y = 1 | X) / P(Y = 2 | X) = \pi_1 * P(X | Y = 1) / (\pi_2 * P(X | Y = 2))$$

So the equality holds.

b) Derive the Decision Rule:

Decision Rule: Assign X to class 1 if $P(Y = 1 | X) > P(Y = 2 | X)$, otherwise assign it to class 2.

If $g(X) = 0$, assign X to either class arbitrarily.

Assign X to class 1 if:

$$\pi_1 * P(X | Y = 1) / (\pi_2 * P(X | Y = 2)) > 1$$

Take logarithm:

$$\log(\pi_1) + \log(P(X | Y = 1)) - \log(\pi_2) - \log(P(X | Y = 2)) > 0$$

Substitute the Multivariate Gaussian Density Function:

$$\log(\pi_1) - \log(\pi_2) + (1/2) * (X - \mu_1)' * \Sigma^{-1} * (X - \mu_1) - (1/2) * (X - \mu_2)' * \Sigma^{-1} * (X - \mu_2) > 0$$

$$\log(\pi_1/\pi_2) + (\mu_1 - \mu_2)' * \Sigma^{-1} * X - (1/2) * (\mu_1' * \Sigma^{-1} * \mu_1 - \mu_2' * \Sigma^{-1} * \mu_2) > 0$$

Define the discriminant function $g(X)$:

$$g(X) = \log(\pi_1/\pi_2) + (\mu_1 - \mu_2)' * \Sigma^{-1} * X - (1/2) * (\mu_1' * \Sigma^{-1} * \mu_1 - \mu_2' * \Sigma^{-1} * \mu_2)$$

Decide:

- Assign X to class 1 if $g(X) > 0$.
- Assign X to class 2 if $g(X) < 0$.

- If $g(X) = 0$, assign X to either class randomly.

4. a) Verify the properties of the Exponential Family Distributions:

Definition of Expectation:

$$E(Y) = \int y f_Y(y; \theta, \phi) dy$$

Substitute pdf/pmf:

$$E(Y) = \int y \exp[y\theta - b(\theta) + c(y, \phi)] / a(\phi) dy$$

Differentiate with respect to θ :

$$\partial/\partial\theta [y\theta - b(\theta) + c(y, \phi)] = y - b'(\theta)$$

Rewrite:

$$E(Y) = \int [\partial/\partial\theta \exp[y\theta - b(\theta) + c(y, \phi)] + b'(\theta) \exp[y\theta - b(\theta) + c(y, \phi)]] / a(\phi) dy$$

The **first term is a constant with derivative wrt. Y , so it will equal zero:**

$$E(Y) = b'(\theta) \int \exp[y\theta - b(\theta) + c(y, \phi)] / a(\phi) dy$$

The **second term is just the integral of the pdf/pmf over its entirety**, since the *sum of all the probabilities must equal 1*, then the second term is:

$$b'(\theta) * 1 = b'(\theta)$$

Thus,

$$E(Y) = b'(\theta)$$

b) Verify Poisson Distribution Parameters:

Given: $Y \sim \text{Poisson}(\mu)$

Show: $\theta = \log(\mu)$, $\phi = 1$, $b(\theta) = \exp(\theta)$, $a(\phi) = \phi$, $c(y, \phi) = -\log(y!)$

Substitute the parameters into the Exponential Family Distribution:

$$f(y; \theta, \phi) = \exp[(y \log(\mu) - \exp(\theta)) / 1 - \log(y!)]$$

Take logarithm:

$$\log f(y; \log(\mu), 1) = y \log(\mu) - \exp(\theta) - \log(y!)$$

Since $\theta = \log(\mu)$, we can substitute $\exp(\theta)$ with μ . This gives us:

$$\log f(y; \log(\mu), 1) = y \log(\mu) - \mu - \log(y!)$$

The above is equal to the *natural log* of the PMF of Poisson which is shown here:

$$P(Y = y) = (\lambda^y * e^{-\lambda}) / y!$$

This is the Poisson pmf with parameters:

- y is the random variable (the number of events)
- μ is the average rate of events
- $\log(y!)$ is the natural logarithm of the factorial of y

Section 2 R code

title: "Mini Project 3"
author: "Benny Frisella"
date: "2024-10-20"
output: pdf_document

````{r setup, include=FALSE}`



```
knitr::opts_chunk$set(root.dir = "C:/Users/benny/Documents/University of Texas @
Dallas/Statistical Learning/Datasets")
```
```

##1. Consider the diabetes dataset from Mini Project 2. Use all predictors for all the models considered for this problem.

(a) Fit a logistic regression model using all predictors in the data. Provide its error rate, sensitivity, and specificity based on training data.

```
```{r}
#(a)
```

```
setwd("~/University of Texas @ Dallas/Statistical Learning/Datasets")
diabetes <- read.csv("diabetes.csv")
summary(diabetes) -> medium-sized set
colnames(diabetes)
logReg <- glm(Outcome ~ Pregnancies.. + Glucose.. + BloodPressure.. + SkinThickness.. +
Insulin.. + BMI.. + DiabetesPedigreeFunction.. + Age.., family = binomial, data = diabetes)

#Randomly split data into: 70% training and 30% prediction sets
#set.seed(8)
sampleSize <- floor(0.7 * nrow(diabetes))
trainIndex <- sample(seq(nrow(diabetes)), size = sampleSize)

#include randomly chosen indices for training
trainData <- diabetes[trainIndex,]

#include what wasn't chosen in training set into test set
testData <- diabetes[-trainIndex,]

#Build model on training set
fit <- glm(Outcome ~ Pregnancies.. + Glucose.. + BloodPressure..
+ SkinThickness.. + Insulin.. + BMI..
+ DiabetesPedigreeFunction.. + Age..,
family = binomial, data = trainData)

summary(fit)

#estimated probabilities for test data
```

```
logReg.prob <- predict(fit, testData, type = "response")
```

```
#predicted classes (using 0.5 cutoff)
```

```
logReg.pred <- ifelse(logReg.prob >= 0.5, 1, 0)
```

```
#test error rate
```

```
1 - mean(logReg.pred == testData[, "Outcome"])
```

```
Confusion matrix and (sensitivity, specificity)
```

```
'+' = 1, '-' = 0
```

```
table(logReg.pred, testData[, "Outcome"])
```

```
errorRate <- (37 + 92) / (353 + 118 + 37 + 92)
```

```
sensitivity <- 118 / (118 + 92)
```

```
specificity <- 353 / (353 + 37)
```

```
cat("\nError rate: (FP + FN) / (TP + TN + FP + FN) =", errorRate)
```

```
cat("\nSensitivity: TP / (TP + FN)", sensitivity)
```

```
cat("\nSpecificity: TN / (TN + FP)", specificity)
```

```
```\n
```

(b) Write your own code to estimate the test error rate of the model in (a) using LOOCV

```
```\n{r}
```

```
##(b)
```

```
#observations
```

```
n <- nrow(diabetes)
```

```
squaredErrorsLoocv <- numeric(n)
```

```
#LOOCV
```

```
for (i in 1:n)
```

```
{
```

```
 #create training/test sets
```

```
 trainData <- diabetes[-i,]
```

```
 testData <- diabetes[i, , drop = FALSE]
```

```
 #train using logistic regression
```

```
 LOOCV <- glm(Outcome ~ Pregnancies.. + Glucose.. + BloodPressure..
```

```
 + SkinThickness.. + Insulin.. + BMI..
```

```
 + DiabetesPedigreeFunction.. + Age..,
```

```

data = train_data, family = binomial)

#make predictions for the i-th test case
prediction <- predict(LOOCV, newdata = testData, type = "response")

#square the difference, store
actualOutcomes <- testData$Outcome
squaredErrorsLoocv[i] <- (prediction - actualOutcomes) ^ 2
}

```

```

#Mean Squared Error for LOOCV
LOOCV_MSE <- mean(squaredErrorsLoocv)

```

LOOCV\_MSE

```

```

```

```

```{r}

```

#(c)

```

library(boot)

```

```

#Build glm model on entire dataset
model <- glm(Outcome ~ Pregnancies.. + Glucose..
 + BloodPressure.. + SkinThickness.. + Insulin..
 + BMI.. + DiabetesPedigreeFunction.. + Age..,
 family = binomial, data = diabetes)

```

```

#k-fold LOOCV special case
KFoldsLOOCV <- cv.glm(diabetes, model, K = n)

```

KFoldsLOOCV

```

```

```

```

```{r}

```

?cv.glm

```

```

```

```

```{r}

```

# k = 10 folds

```

KFolds10 <- cv.glm(diabetes, model, K = 10)

```

KFolds10

```

```

```

##2. Consider the oxygen saturation data stored in oxygen_saturation.txt file. The data consist of measurements of percent saturation of hemoglobin with oxygen in 72 adults, obtained using an oxygen saturation monitor (OSM, method 1) and a pulse oximetry screener (POS, method 2).

(a) Let Y_1 and Y_2 denote the population of observations of methods 1 and 2, respectively, and $D = Y_1 - Y_2$ denote their difference. Let θ be the total deviation index (TDI) between the two methods. For a given large probability p , it is defined as the p th quantile of $|D|$. Here we will take $p = 0.90$. Argue that smaller values for θ imply better agreement.

```
```{r}
```

```
##(a)
```

```
setwd("~/University of Texas @ Dallas/Statistical Learning/Datasets")
oxygenSats <- read.csv("oxygenSaturation.txt", sep = "\t", header = TRUE)
```

```
oxygenSats$D <- abs(oxygenSats$pos - oxygenSats$osm)
```

```
hist(oxygenSats$D, main = "Total Deviation Index Distribution", xlab = "Differences Between
POS and OSM")
```

```
```
```

(b) Provide a point estimate $\hat{\theta}$ of θ . (Tip: If the population parameter is a quantile, what should be its natural estimator?) Write your own code to compute (nonparametric) bootstrap estimates of bias and standard error of $\hat{\theta}$, and a 95% upper confidence bound for θ computed using the percentile method. Interpret the results.

```
```{r}
```

```
##(b)
```

```
#set.seed(8)
```

```
#point-estimation
thetaHat <- quantile(oxygenSats$D, 0.9)
```

```
Bootstrap NonParametric
B <- 1000 #number of samples
bootstrapEstimates <- numeric(B)
```

```

for (b in 1:B)
{
 bootstrapSample <- sample(oxygenSats$D, replace = TRUE)
 bootstrapEstimates[b] <- quantile(bootstrapSample, 0.9)
}

```

```

#bias
bias <- mean(bootstrapEstimates) - thetaHat

```

```

#standard error
sdError <- sd(bootstrapEstimates)

```

```

95% upper confidence bound
upperBound <- quantile(bootstrapEstimates, 0.95)

```

```

cat("\nPoint estimate (theta_hat):", thetaHat, "\n")
cat("Bias:", bias, "\n")
cat("Standard Error:", sdError, "\n")
cat("95% Upper Confidence Bound:", upperBound, "\n")
```

```

(c) Repeat the computation in (d) using boot package in R, or check bootstrap function from the SciPy library (scipy.stats.bootstrap) in Python, and compare your results.

```

```{r}
#(c)
library(boot)

statistic <- function(data, indices)
{
 quantile(abs(data[indices, 1] - data[indices, 2]), 0.9)
}

```

```

bootstrap <- boot(oxygenSats, statistic, R = 1000)

```

```

bootstrap
```

```